Lab 3

Working with Git History, Stashing, and Reverting Commits

DevOps F26

Name	Mehmoona Abid
Roll number	2023-BCS-097
Section	A
GitHub repository link	https://github.com/mehmoonaabid/DevOps

Submission details

Repository: DevOps

Directory: Labs/Lab03

Screenshots: Labs/Lab03/screenshots

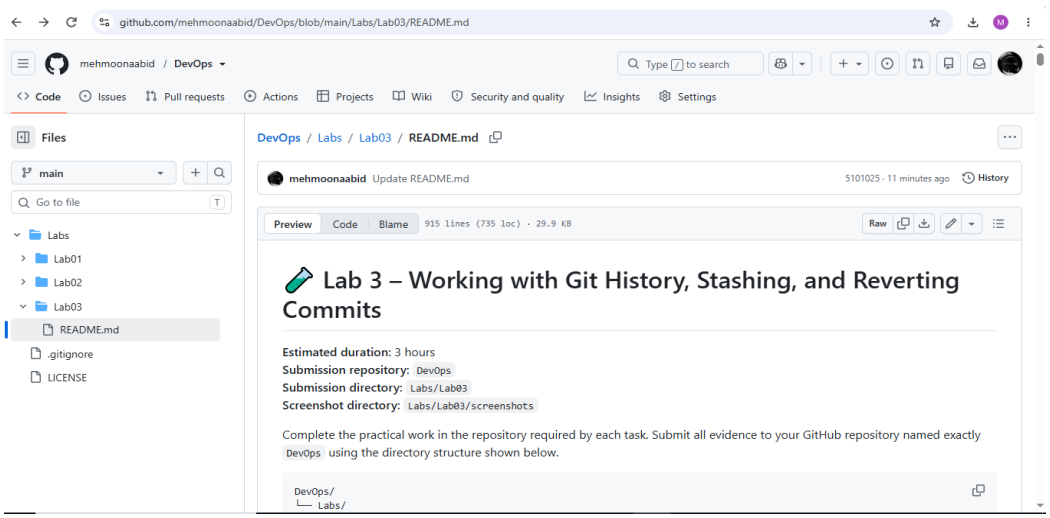
Task 1 – Handling Local and Remote Commit Conflicts (Pull vs Pull--rebase)

This task demonstrates what happens when your local repository and remote repository both have new commits — and how git pull and git pull --rebase behave differently.

Steps

1. Open your repository on **GitHub** and edit the README.md file directly in the browser.
    - Add a new line such as:
    - This line was added remotely from GitHub.
    - Commit your change on GitHub (e.g., message: *Remote update to README*).

- Save a screenshot of your browser showing the change as **remote_edit.png.**

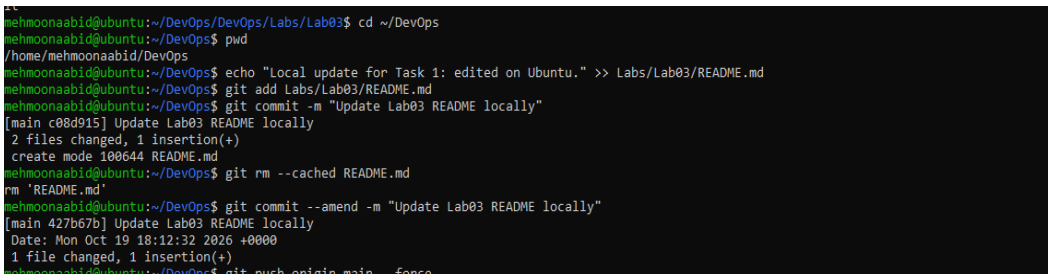


2. On your **local machine**, open the same repository folder.
   - Edit the same README.md file locally, for example: (Avoid merge conflict)
   - This line was added locally.
   - Stage and commit your change:
   - git add README.md

   git commit -m "Local update to README"

   - Save a screenshot of your terminal as

     local_commit.png.



3. Try to push:git push origin main

You'll see an error message similar to:

! [rejected] main -> main (fetch first)

error: failed to push some refs to 'github.com:username/repo.git'

hint: Updates were rejected because the remote contains work that you do not have locally.

- Save a screenshot of this error as

  push_error.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git push origin main
warning: redirecting to https://github.com/mehmoonaabid/DevOps.git/
To https://github.com/mehmoonaabid/DevOps.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/mehmoonaabid/DevOps.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
mehmoonaabid@ubuntu:~/DevOps$
```

4. To fix this, pull the latest changes from remote:

git pull --no-rebase origin main

- Git will **merge** your local and remote commits, creating a *merge commit*.
- Save a screenshot as

  merge_commit.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git pull --no-rebase origin main
warning: redirecting to https://github.com/mehmoonaabid/DevOps.git/
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 1.12 KiB | 384.00 KiB/s, done.
From https://github.com/mehmoonaabid/DevOps
 * branch            main       -> FETCH_HEAD
   68df598..b0a2ace  main       -> origin/main
Auto-merging Labs/Lab03/README.md
Merge made by the 'ort' strategy.
 Labs/Lab03/README.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
mehmoonaabid@ubuntu:~/DevOps$
```

5. Push again:

git push -u origin main

- Save a screenshot as

  push_after_merge.png

```
mehmoonaabid@ubuntu:~/DevOps$ git push -u origin main
warning: redirecting to https://github.com/mehmoonaabid/DevOps.git/
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 2 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (10/10), 1.04 KiB | 532.00 KiB/s, done.
Total 10 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 2 local objects.
To https://github.com/mehmoonaabid/DevOps.git
   b0a2ace..51d51fc  main -> main
branch 'main' set up to track 'origin/main'.
mehmoonaabid@ubuntu:~/DevOps$
```

6. Now repeat the same process, but this time use **rebase instead of merge**:
    - Make another remote edit on GitHub in README.md.
    - Then make another local edit and commit it.
    - Instead of using git pull, run:

git pull --rebase origin main

- Git will **replay your local commits on top** of the pulled commits, keeping history linear.
- Save a screenshot as

  rebase_pull.png

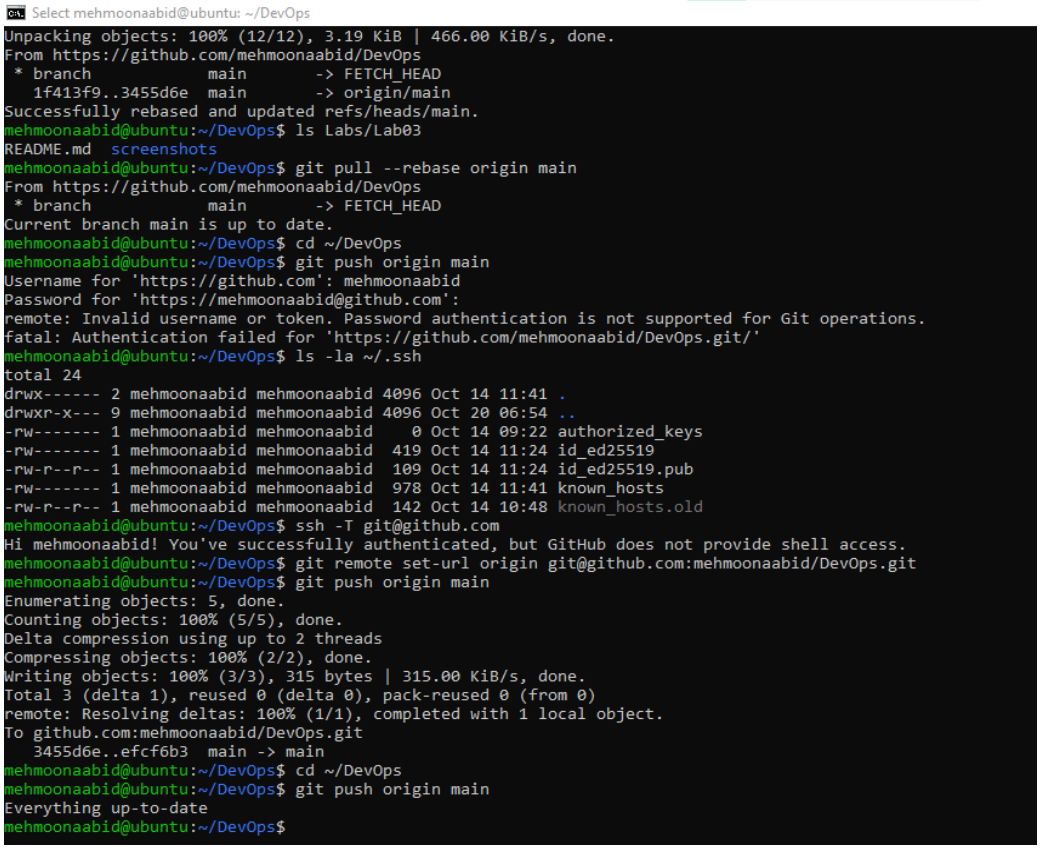
```
mehmoonaabid@ubuntu:~/DevOps$ nano README.md
mehmoonaabid@ubuntu:~/DevOps$ git add README.md
date to README"mehmoonaabid@ubuntu:~/DevOps$ git commit -m "Second local update to README"
[main a3cb3e7] Second local update to README
 1 file changed, 1 insertion(+)
mehmoonaabid@ubuntu:~/DevOps$ git pull --rebase origin main
From https://github.com/mehmoonaabid/DevOps
 * branch            main       -> FETCH_HEAD
Current branch main is up to date.
mehmoonaabid@ubuntu:~/DevOps$
```

- Push again:

git push -u origin main

- Save a screenshot as

push_after_rebase.png



📸 **Screenshot Required:**

- remote_edit.png
- local_commit.png
- push_error.png
- merge_commit.png
- push_after_merge.png
- rebase_pull.png
- push_after_rebase.png

Summary:

- git pull --no-rebase merges remote changes with your local commits, possibly creating a merge commit.

- git pull --rebase keeps commit history linear by replaying your local commits on top of the updated remote branch.
- Both are ways to resolve push conflicts when your local repo is behind the remote.

Task 2 – Creating and Resolving Merge Conflicts Manually

This task will help you understand how Git handles file conflicts when two people modify the same line of code differently.

Steps

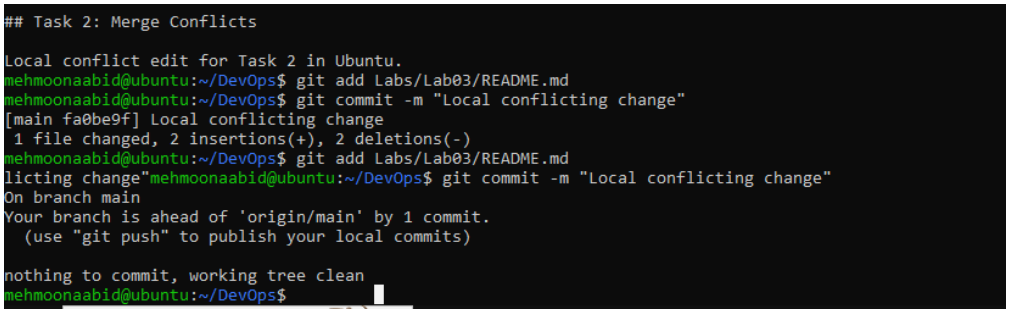
1. On **GitHub (remote)**, open your README.md file and change an existing line.
   For example, if it currently says:
2. This line was added remotely from GitHub.

   Change it to:

   This line was updated remotely again.

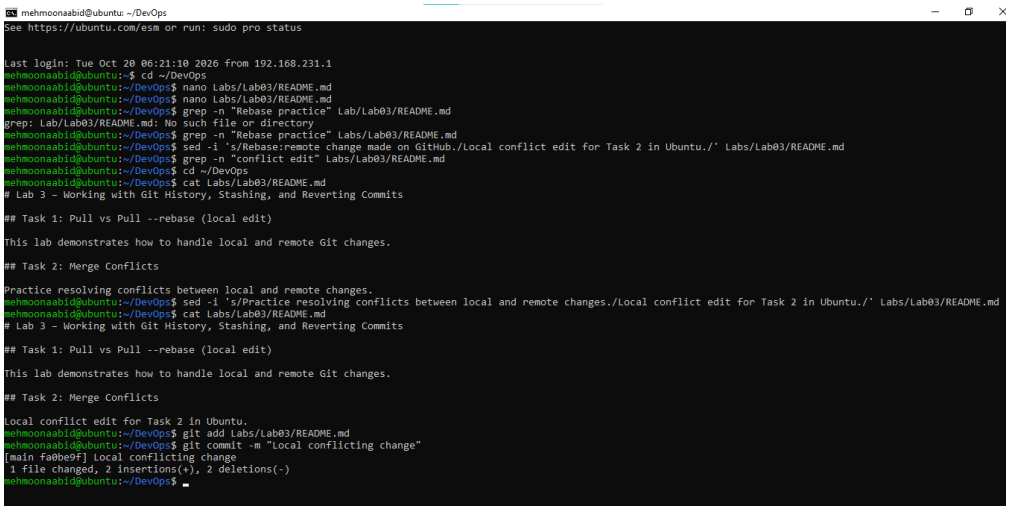
   - Commit your change with the message: *Remote conflicting change*
   - Save a screenshot as

     remote_conflict_edit.png.



3. On your **local machine**, edit the **same line** in the same file but make a **different change**:
4. This line was updated locally at the same time.

   - Save a screenshot of your edit in VS Code or nano or Vim as **local_conflict_edit.png**.

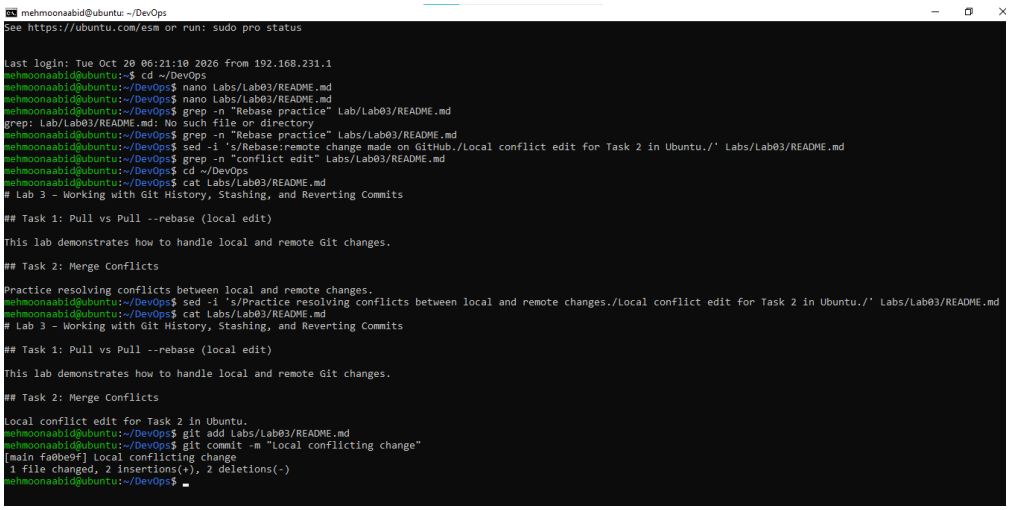


5. Stage and commit your local change:
6. git add README.md

   git commit -m "Local conflicting change"

   - Save a screenshot as

     local_conflict_commit.png.

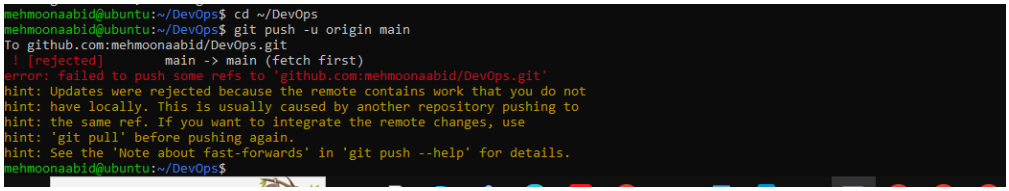


7. Try to push:

   git push -u origin main

   - The push will be **rejected**, since the remote version has conflicting changes.
   - Save a screenshot of this error as

     conflict_push_error.png.

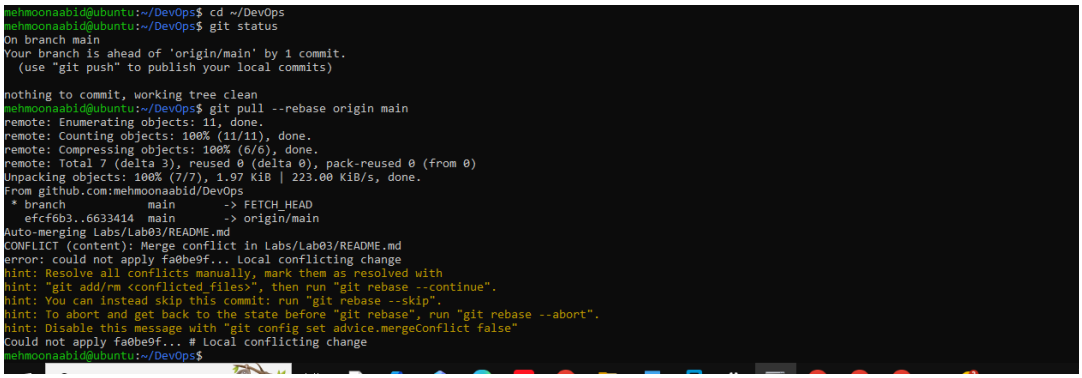


8. Pull with rebase to bring in remote changes:

   git pull --rebase origin main

   - Git will **pause** the rebase and notify you about a conflict.
   - Save a screenshot of the conflict message as

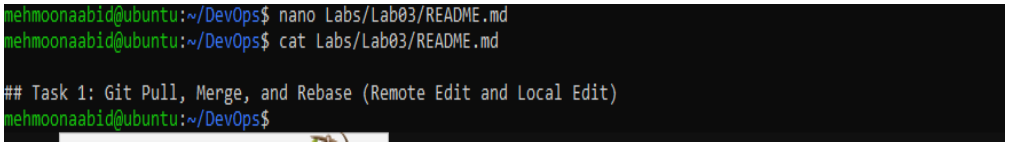
     conflict_message.png.



9. Open the README.md file in your editor — you'll see conflict markers like this:
10. <<<<<<< HEAD
11. This line was updated locally at the same time.
12. =======
13. This line was updated remotely again.
14. >>>>>>> origin/main

    - Manually edit the file to keep the correct line (for example, merge both ideas).
    - Save a screenshot of the resolved file as

      resolved_readme.png.

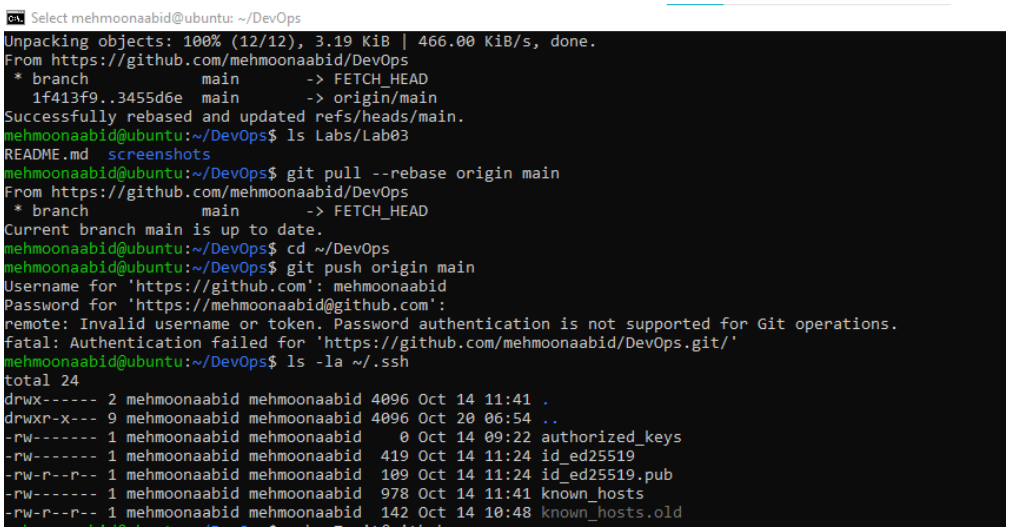


15. After fixing the file, mark the conflict as resolved and continue the rebase:
16. git add README.md

    git rebase --continue

    - Save a screenshot of this step as

      rebase_continue.png.

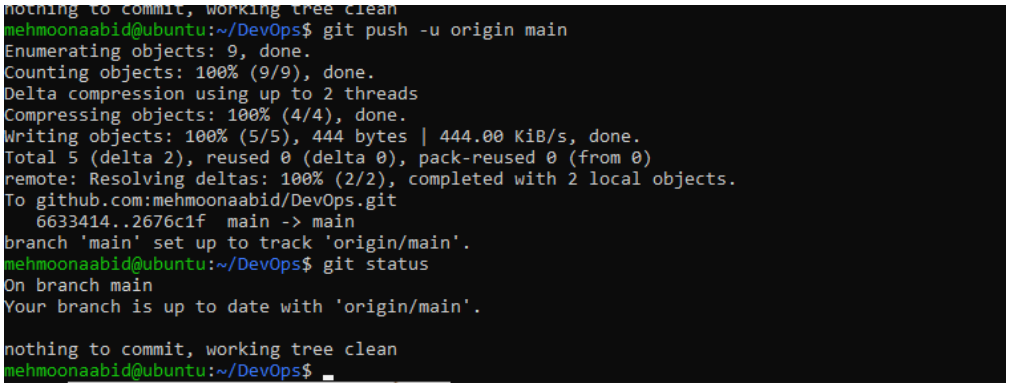


17. Finally, push your changes:

    git push -u origin main

    - Save a screenshot as

      push_after_resolve.png.



📸 **Screenshot Required:**

- remote_conflict_edit.png
- local_conflict_edit.png

- local_conflict_commit.png
- conflict_push_error.png
- conflict_message.png
- resolved_readme.png
- rebase_continue.png
- push_after_resolve.png

Summary:

- When two people edit the same line, Git will pause and ask for manual conflict resolution.
- Conflict markers appear in the file and you must choose what to keep.
- After resolving, continue the rebase and push to remote.
- This process ensures both sets of changes are handled safely.

Task 3 – Managing Ignored Files with .gitignore and Removing Tracked Files

In this task, you will learn how to use a .gitignore file to exclude files or directories from version control and how to remove them from Git's tracking while keeping them locally.

Steps

1. Create a new folder named textfiles inside your repository:

   mkdir textfiles

2. Inside the textfiles folder, create three text files:
3. echo "File A content" > textfiles/a.txt
4. echo "File B content" > textfiles/b.txt

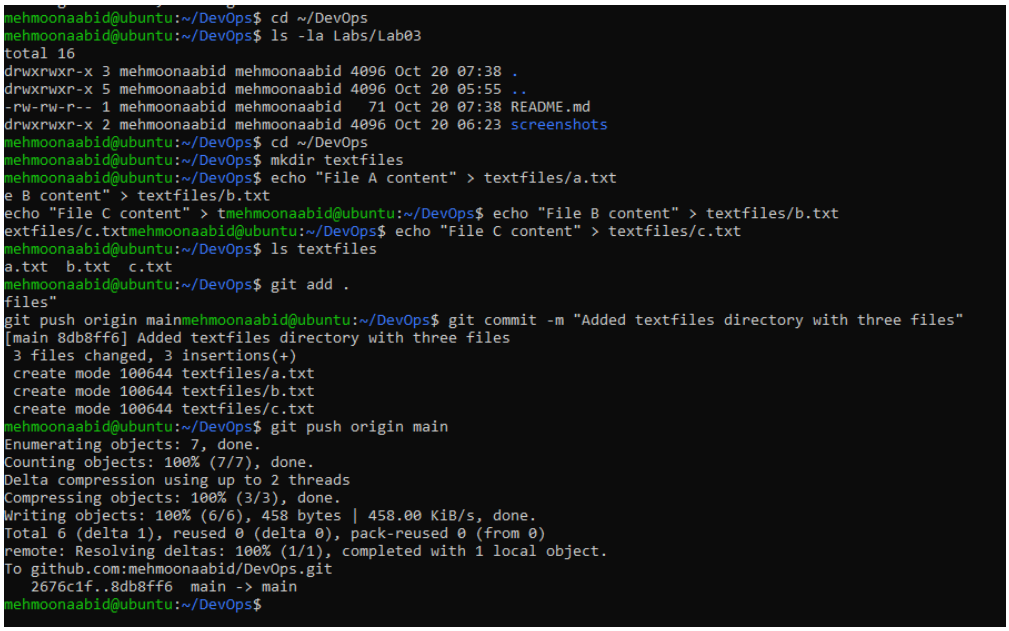
   echo "File C content" > textfiles/c.txt

5. Add and commit the new directory:
6. git add .

7. git commit -m "Added textfiles directory with three files"

   git push origin main

   - Save a screenshot as **push_textfiles.png**.



8. Now, create a .gitignore file in the root of your repository:

   echo "textfiles/*" > .gitignore

9. Add and commit the .gitignore file:

10. git add .gitignore

11. git commit -m "Added .gitignore to ignore textfiles directory"

    git push origin main

    - Save a screenshot as

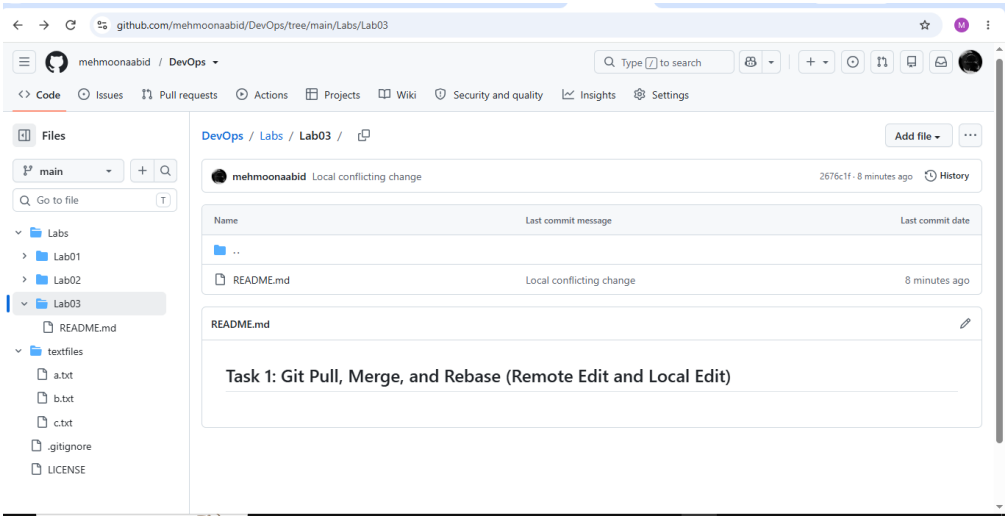
      gitignore_push.png.



12. Go to **GitHub** and check your repository — notice that the textfiles directory is **still visible** on the remote.

    - Save a screenshot as

      repo_still_has_textfiles.png.



13. To remove already tracked files from Git (but not from your local system), run:

    git rm -r --cached textfiles

14. Commit and push again:

15. git add .

16. git commit -m "Removed tracked textfiles directory"

    git push origin main

    - Save a screenshot as

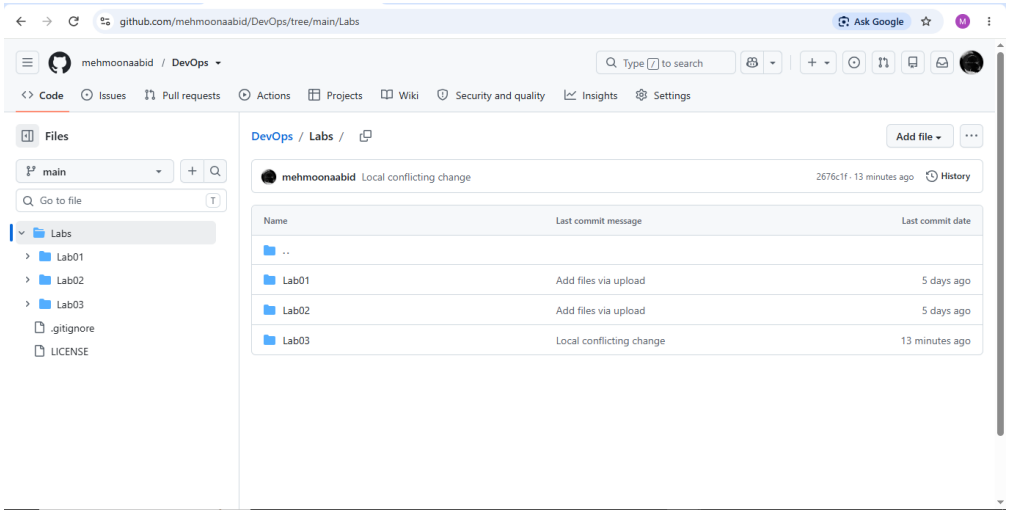
      rm_cached_push.png.

```
mehmoonaabid@ubuntu:~/DevOps$ cd ~/DevOps
mehmoonaabid@ubuntu:~/DevOps$ git rm -r --cached textfiles
rm 'textfiles/a.txt'
rm 'textfiles/b.txt'
rm 'textfiles/c.txt'
mehmoonaabid@ubuntu:~/DevOps$ git add .
mehmoonaabid@ubuntu:~/DevOps$ git commit -m "Removed tracked textfiles directory"
[main b7b6f24] Removed tracked textfiles directory
 3 files changed, 3 deletions(-)
 delete mode 100644 textfiles/a.txt
 delete mode 100644 textfiles/b.txt
 delete mode 100644 textfiles/c.txt
mehmoonaabid@ubuntu:~/DevOps$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 248 bytes | 124.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:mehmoonaabid/DevOps.git
   a5d52f5..b7b6f24  main -> main
mehmoonaabid@ubuntu:~/DevOps$
```

17. Check your GitHub repository again — the textfiles folder should now be **deleted remotely**.

    - Save a screenshot as

      repo_textfiles_removed.png.



📸 Screenshot Required:

- push_textfiles.png
- gitignore_push.png
- repo_still_has_textfiles.png
- rm_cached_push.png
- repo_textfiles_removed.png

Summary:

- .gitignore prevents new files from being tracked, but does not remove ones already tracked.
- To untrack files that are already committed, use git rm --cached.
- This lets you keep files locally but remove them from the remote repo.

Task 4 – Create Temporary Changes and Use git stash

This task demonstrates how git stash allows you to temporarily save uncommitted changes so that you can safely switch branches without losing work.

Steps

1. Create a feature-branch.

   git checkout -b feature-branch

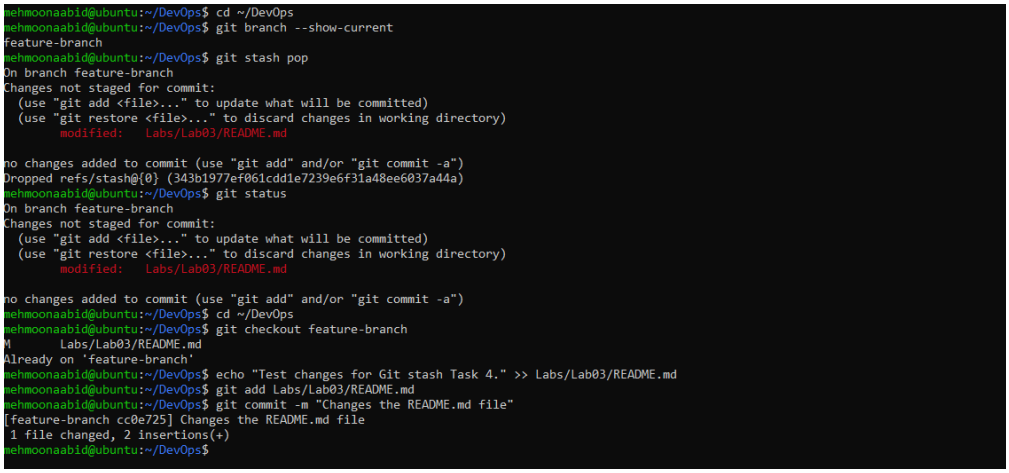
2. Modify any file (for example, README.md) by adding a few test lines.

3. git add README.md

   git commit -m "Changes the README.md file"

   - Save a screenshot as

     modified_readme.png.



4. Modify any file (for example, README.md) by adding a few new lines.

5. Without committing or stashing the changes, try to switch to another branch:

   git checkout main

   You'll see an error message similar to:

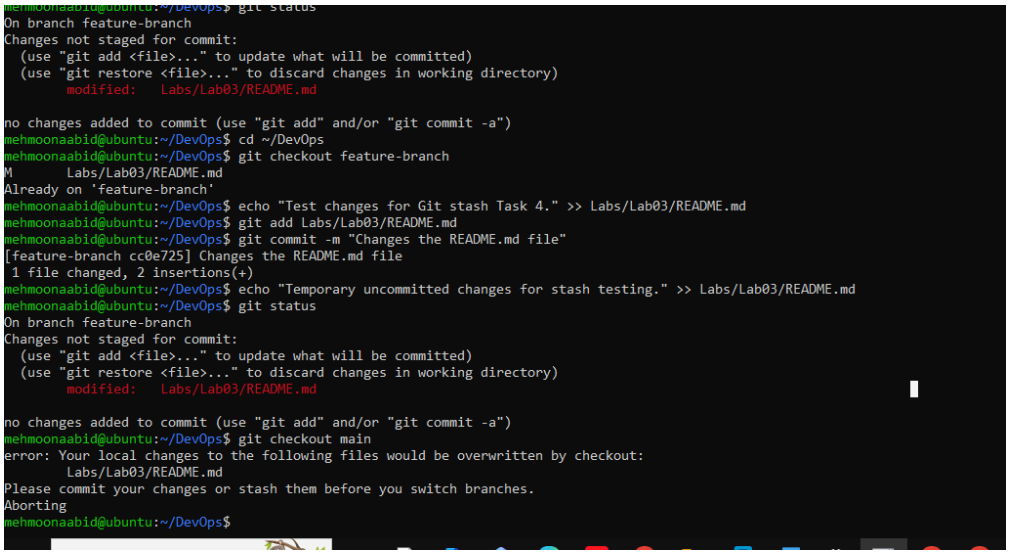
   error: Your local changes to the following files would be overwritten by checkout:

   README.md

Please commit your changes or stash them before you switch branches.

- Save a screenshot as

  checkout_error.png.

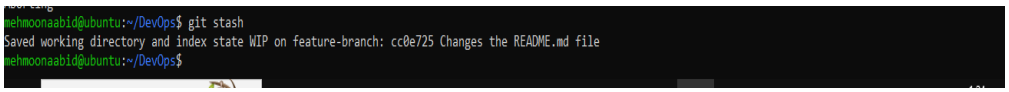


6. To fix this, **stash your changes**:

   git stash

   - Save a screenshot as

     stash_command.png.

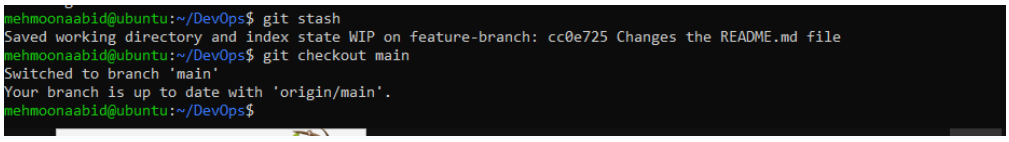


7. Try switching branches again — it should now work:

   git checkout main

   - Save a screenshot as

     branch_switched.png.



8. Return to your previous branch (for example, feature-branch):

   git checkout feature-branch

   - Save a screenshot as back_to_feature.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git checkout feature-branch
Switched to branch 'feature-branch'
mehmoonaabid@ubuntu:~/DevOps$
```

9. Check your working directory status:

   git status

   It should show:

   nothing to commit, working tree clean

   - Save a screenshot as status_clean.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git status
On branch feature-branch
nothing to commit, working tree clean
mehmoonaabid@ubuntu:~/DevOps$
```

10. Now restore your stashed changes:

    git stash pop

    You'll see a message indicating that changes were reapplied:

    On branch feature-branch

    Changes not staged for commit:

    (use "git add <file>..." to update what will be committed)

    (use "git restore <file>..." to discard changes in working directory)

    - Save a screenshot as stash_pop.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git stash
Saved working directory and index state WIP on feature-branch: cc0e725 Changes the README.md file
mehmoonaabid@ubuntu:~/DevOps$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
mehmoonaabid@ubuntu:~/DevOps$ git checkout feature-branch
Switched to branch 'feature-branch'
mehmoonaabid@ubuntu:~/DevOps$ git status
On branch feature-branch
nothing to commit, working tree clean
mehmoonaabid@ubuntu:~/DevOps$ git stash pop
On branch feature-branch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Labs/Lab03/README.md

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (d006fe762240eecc65077292d8f51d793e7bb79b)
mehmoonaabid@ubuntu:~/DevOps$ git status
On branch feature-branch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Labs/Lab03/README.md

no changes added to commit (use "git add" and/or "git commit -a")
mehmoonaabid@ubuntu:~/DevOps$
```

11. Confirm that your previous edits are back in the file.

📸 **Screenshot Required:**

- modified_readme.png
- checkout_error.png
- stash_command.png
- branch_switched.png
- back_to_feature.png
- status_clean.png
- stash_pop.png

Summary:

- git stash saves your uncommitted changes, letting you switch branches safely.
- You can restore stashed changes anytime with git stash pop.
- This is useful for quickly saving work-in-progress without cluttering commit history.

Task 5 – Checkout a Specific Commit Using git log

1. View commit history:

   git log --oneline

   - Save a screenshot as log_before_checkout.png.

```
mehmoonaabid@ubuntu:~/DevOps$ cd ~/DevOps
mehmoonaabid@ubuntu:~/DevOps$ git checkout main
error: Your local changes to the following files would be overwritten by checkout:
        Labs/Lab03/README.md
Please commit your changes or stash them before you switch branches.
Aborting
mehmoonaabid@ubuntu:~/DevOps$ git log --oneline
cc0e725 (HEAD -> feature-branch) Changes the README.md file
b7b6f24 (origin/main, origin/HEAD, main) Removed tracked textfiles directory
a5d52f5 Added .gitignore to ignore textfiles directory
8db8ff6 Added textfiles directory with three files
2676c1f Local conflicting change
6633414 Delete README.md
e2a6e2f Remote conflicting change
efcf6b3 Second local update to README
3455d6e Update README.md
dc380eb Add files via upload
6b97200 Delete Labs/Lab03/README.md
1f413f9 Update README.md
21f5fbd Update README.md
8736292 Update README.md
3dca27a Update README.md
e350c87 Update README.md
0f9fc66 Local update for rebase
ec5d459 Update README.md
51d51fc Merge branch 'main' of https://github.com/mehmoonaabid/DevOps
b0a2ace Update README.md
47b6c59 Local edit for auto merge test
68df598 Update README.md
3a3b8d4 Local update to README
7bdc9ca Merge remote and local README updates
2437c2a Merge remote and local README updates
d531a0c Local edit for rebase test
5c673c8 Update README.md
06820b7 Resolved merge conflict
8e58428 Updated Lab03 README
427b67b Update Lab03 README locally
5101025 Update README.md
bdf6e14 Added Lab03 folder
383940d Delete Lab03
```

2. Copy any previous commit hash (e.g., a1b2c3d).
3. Checkout that commit:

   git checkout <commit-hash>

   - Save a screenshot as detached_head.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git status
On branch feature-branch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Labs/Lab03/README.md

no changes added to commit (use "git add" and/or "git commit -a")
mehmoonaabid@ubuntu:~/DevOps$ git stash push -m "Save Lab03 README changes"
Saved working directory and index state On feature-branch: Save Lab03 README changes
mehmoonaabid@ubuntu:~/DevOps$ git stash list
stash@{0}: On feature-branch: Save Lab03 README changes
mehmoonaabid@ubuntu:~/DevOps$ git checkout a5d52f5
Note: switching to 'a5d52f5'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at a5d52f5 Added .gitignore to ignore textfiles directory
mehmoonaabid@ubuntu:~/DevOps$
```

4. To return to your main branch:

   git checkout main

   - Save a screenshot as back_to_main.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git checkout main
Previous HEAD position was a5d52f5 Added .gitignore to ignore textfiles directory
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
mehmoonaabid@ubuntu:~/DevOps$
```

📸 **Screenshot Required:**

- log_before_checkout.png
- detached_head.png
- back_to_main.png

Summary:

- You can revisit past versions by checking out a specific commit hash.
- In detached HEAD state, changes are not attached to any branch.
- Always return to your main branch after inspecting old commits.

Task 6 – Resetting Commits (Soft vs Hard Reset) (With Verification Steps)

This task demonstrates the difference between a soft and hard reset in Git. In this version, you will **verify the presence of changes in files and confirm the existence of commits after each reset**.

Steps

1. **Add a new line in any file and commit it:**
2. # Edit any file (e.g., README.md), add a line
3. git add .

   git commit -m "Added test line"

   - Save a screenshot as first_commit.png.

```
mehmoonaabid@ubuntu:~/DevOps$ cd ~/DevOps
mehmoonaabid@ubuntu:~/DevOps$ git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
mehmoonaabid@ubuntu:~/DevOps$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
mehmoonaabid@ubuntu:~/DevOps$ git stash push -m "Save changes before Task 6"
main
git statusNo local changes to save
mehmoonaabid@ubuntu:~/DevOps$ git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
mehmoonaabid@ubuntu:~/DevOps$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
mehmoonaabid@ubuntu:~/DevOps$ echo "Task 6 first test line." >> Labs/Lab03/README.md
mit -m "mehmoonaabid@ubuntu:~/DevOps$ git add Labs/Lab03/README.md
Added test line"mehmoonaabid@ubuntu:~/DevOps$ git commit -m "Added test line"
[main 59eaa47] Added test line
 1 file changed, 1 insertion(+)
mehmoonaabid@ubuntu:~/DevOps$
```

4. **Add another change and commit again:**
5. # Edit the file again, add a different line
6. git add .

   git commit -m "Second test commit"

   - Save a screenshot as second_commit.png.

```
mehmoonaabid@ubuntu:~/DevOps$ echo "Task 6 first test line." >> Labs/Lab03/README.md
mit -m "mehmoonaabid@ubuntu:~/DevOps$ git add Labs/Lab03/README.md
Added test line"mehmoonaabid@ubuntu:~/DevOps$ git commit -m "Added test line"
[main 59eaa47] Added test line
 1 file changed, 1 insertion(+)
mehmoonaabid@ubuntu:~/DevOps$ echo "Task 6 second test line." >> Labs/Lab03/README.md
mehmoonaabid@ubuntu:~/DevOps$ git add Labs/Lab03/README.md
mehmoonaabid@ubuntu:~/DevOps$ git commit -m "Second test commit"
[main 458f57b] Second test commit
 1 file changed, 1 insertion(+)
mehmoonaabid@ubuntu:~/DevOps$
```

7. **View the history before reset:**

   git log --oneline

   - Save a screenshot as log_before_reset.png.

```
mehmoonaabid@ubuntu: ~/DevOps
mehmoonaabid@ubuntu:~/DevOps$ git commit -m "Second test commit"
[main 458f57b] Second test commit
 1 file changed, 1 insertion(+)
mehmoonaabid@ubuntu:~/DevOps$ git log --oneline
458f57b (HEAD -> main) Second test commit
59eaa47 Added test line
b7b6f24 (origin/main, origin/HEAD) Removed tracked textfiles directory
a5d52f5 Added .gitignore to ignore textfiles directory
8db8ff6 Added textfiles directory with three files
2676c1f Local conflicting change
6633414 Delete README.md
e2a6e2f Remote conflicting change
efcf6b3 Second local update to README
3455d6e Update README.md
dc380eb Add files via upload
6b97200 Delete Labs/Lab03/README.md
1f413f9 Update README.md
21f5fbd Update README.md
8736292 Update README.md
3dca27a Update README.md
e350c87 Update README.md
0f9fc66 Local update for rebase
ec5d459 Update README.md
51d51fc Merge branch 'main' of https://github.com/mehmoonaabid/DevOps
b0a2ace Update README.md
47b6c59 Local edit for auto merge test
68df598 Update README.md
3a3b8d4 Local update to README
7bdc9ca Merge remote and local README updates
2437c2a Merge remote and local README updates
d531a0c Local edit for rebase test
5c673c8 Update README.md
06820b7 Resolved merge conflict
8e58428 Updated Lab03 README
427b67b Update Lab03 README locally
5101025 Update README.md
bdf6e14 Added Lab03 folder
383940d Delete Lab03
cc3908b Create Lab03
45e774a Delete Labs/README.md
fc414aa Create README.md
```

8. **Check file contents for both changes:**
    - Open the edited file (e.g., README.md) and visually confirm BOTH added lines exist.
    - Save a screenshot as file_before_reset.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git log --oneline -3
458f57b (HEAD -> main) Second test commit
59eaa47 Added test line
b7b6f24 (origin/main, origin/HEAD) Removed tracked textfiles directory
mehmoonaabid@ubuntu:~/DevOps$ tail -n 5 Labs/Lab03/README.md

## Task 1: Git Pull, Merge, and Rebase (Remote Edit and Local Edit)
Task 6 first test line.
Task 6 second test line.
mehmoonaabid@ubuntu:~/DevOps$
```

9. **Perform a soft reset (keeps changes in working directory):**

   git reset --soft HEAD~1

    - Save a screenshot as soft_reset.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git log --oneline -3
458f57b (HEAD -> main) Second test commit
59eaa47 Added test line
b7b6f24 (origin/main, origin/HEAD) Removed tracked textfiles directory
mehmoonaabid@ubuntu:~/DevOps$ tail -n 5 Labs/Lab03/README.md

## Task 1: Git Pull, Merge, and Rebase (Remote Edit and Local Edit)
Task 6 first test line.
Task 6 second test line.
mehmoonaabid@ubuntu:~/DevOps$ git reset --soft HEAD~1
mehmoonaabid@ubuntu:~/DevOps$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   Labs/Lab03/README.md

mehmoonaabid@ubuntu:~/DevOps$ git reset --soft HEAD~1
mehmoonaabid@ubuntu:~/DevOps$
```

10. **Check commit history after soft reset:**

    git log --oneline

    - Save a screenshot as log_after_soft_reset.png.

```
mehmoonaabid@ubuntu:~/DevOps$ tail -n 5 Labs/Lab03/README.md

## Task 1: Git Pull, Merge, and Rebase (Remote Edit and Local Edit)
Task 6 first test line.
Task 6 second test line.
mehmoonaabid@ubuntu:~/DevOps$ git log --oneline -3
b7b6f24 (HEAD -> main, origin/main, origin/HEAD) Removed tracked textfiles directory
a5d52f5 Added .gitignore to ignore textfiles directory
8db8ff6 Added textfiles directory with three files
mehmoonaabid@ubuntu:~/DevOps$
```

11. **Verify changes in the file:**

    - Open the file again and confirm both edits are still present (nothing lost).
    - Save a screenshot as file_after_soft_reset.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git log --oneline -3
b7b6f24 (HEAD -> main, origin/main, origin/HEAD) Removed tracked textfiles directory
a5d52f5 Added .gitignore to ignore textfiles directory
8db8ff6 Added textfiles directory with three files
mehmoonaabid@ubuntu:~/DevOps$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   Labs/Lab03/README.md

mehmoonaabid@ubuntu:~/DevOps$ tail -n 5 Labs/Lab03/README.md

## Task 1: Git Pull, Merge, and Rebase (Remote Edit and Local Edit)
Task 6 first test line.
Task 6 second test line.
mehmoonaabid@ubuntu:~/DevOps$
```

12. **Check git status:**

git status

- You should see your changes are staged (ready to commit again).

13. **Perform commit**

    git commit -m "Second Test commit"

    - Save a screenshot as file_before_hard_reset.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git commit -m "Second Test commit"
[main d59f970] Second Test commit
 1 file changed, 2 insertions(+)
mehmoonaabid@ubuntu:~/DevOps$ git log --oneline -3
d59f970 (HEAD -> main) Second Test commit
b7b6f24 (origin/main, origin/HEAD) Removed tracked textfiles directory
a5d52f5 Added .gitignore to ignore textfiles directory
mehmoonaabid@ubuntu:~/DevOps$ git reset --hard HEAD~1
HEAD is now at b7b6f24 Removed tracked textfiles directory
mehmoonaabid@ubuntu:~/DevOps$
```

14. **Perform a hard reset (discards changes completely):**

    git reset --hard HEAD~1

- Save a screenshot as hard_reset.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git commit -m "Second Test commit"
[main d59f970] Second Test commit
 1 file changed, 2 insertions(+)
mehmoonaabid@ubuntu:~/DevOps$ git log --oneline -3
d59f970 (HEAD -> main) Second Test commit
b7b6f24 (origin/main, origin/HEAD) Removed tracked textfiles directory
a5d52f5 Added .gitignore to ignore textfiles directory
mehmoonaabid@ubuntu:~/DevOps$ git reset --hard HEAD~1
HEAD is now at b7b6f24 Removed tracked textfiles directory
mehmoonaabid@ubuntu:~/DevOps$
```

11. **Check commit history after hard reset:**

    git log --oneline

    - Save a screenshot as log_after_hard_reset.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git log --oneline -3
d59f970 (HEAD -> main) Second Test commit
b7b6f24 (origin/main, origin/HEAD) Removed tracked textfiles directory
a5d52f5 Added .gitignore to ignore textfiles directory
mehmoonaabid@ubuntu:~/DevOps$ git reset --hard HEAD~1
HEAD is now at b7b6f24 Removed tracked textfiles directory
mehmoonaabid@ubuntu:~/DevOps$ git log --oneline -3
b7b6f24 (HEAD -> main, origin/main, origin/HEAD) Removed tracked textfiles directory
a5d52f5 Added .gitignore to ignore textfiles directory
8db8ff6 Added textfiles directory with three files
mehmoonaabid@ubuntu:~/DevOps$ it status
E.mdit: command not found
mehmoonaabid@ubuntu:~/DevOps$ tail -n 5 Labs/Lab03/README.md

## Task 1: Git Pull, Merge, and Rebase (Remote Edit and Local Edit)
mehmoonaabid@ubuntu:~/DevOps$
```

12. **Verify changes in the file:**

- Open the file and confirm the **second edit is gone** (reverted to previous commit).
- Save a screenshot as file_after_hard_reset.png.

```
mehmoonaabid@ubuntu:~/DevOps$ git log --oneline -3
d59f970 (HEAD -> main) Second Test commit
b7b6f24 (origin/main, origin/HEAD) Removed tracked textfiles directory
a5d52f5 Added .gitignore to ignore textfiles directory
mehmoonaabid@ubuntu:~/DevOps$ git reset --hard HEAD~1
HEAD is now at b7b6f24 Removed tracked textfiles directory
mehmoonaabid@ubuntu:~/DevOps$ git log --oneline -3
b7b6f24 (HEAD -> main, origin/main, origin/HEAD) Removed tracked textfiles directory
a5d52f5 Added .gitignore to ignore textfiles directory
8db8ff6 Added textfiles directory with three files
mehmoonaabid@ubuntu:~/DevOps$ it status
E.mdit: command not found
mehmoonaabid@ubuntu:~/DevOps$ tail -n 5 Labs/Lab03/README.md

## Task 1: Git Pull, Merge, and Rebase (Remote Edit and Local Edit)
mehmoonaabid@ubuntu:~/DevOps$
```

13. **Check git status:**

    git status

    - Should report "nothing to commit, working tree clean".

Screenshot Required:

- first_commit.png
- second_commit.png
- log_before_reset.png
- file_before_reset.png
- soft_reset.png
- log_after_soft_reset.png
- file_after_soft_reset.png
- file_before_hard_reset.png
- hard_reset.png
- log_after_hard_reset.png
- file_after_hard_reset.png

Summary:

- Soft reset keeps your changes in the file (you can recommit them).

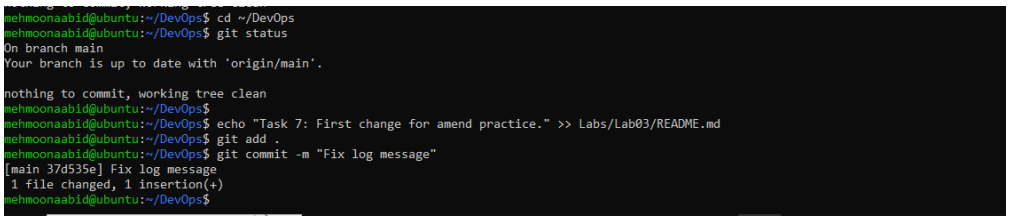
- Hard reset removes the latest commit AND the associated changes from your files.
- Always check both the commit history and your file contents after resetting to understand what was kept and what was discarded.

Task 7 – Amending the Last Commit

1. Make a small change in any file.
2. Stage it and commit:
3. git add .

   git commit -m "Fix log message"

   - Save a screenshot as first_amend_commit.png.



4. Realize you forgot to change another file. Update it now.
5. Add it and amend the last commit:
6. git add .

   git commit --amend

   - Save a screenshot as amend_commit.png.



📸 **Screenshot Required:**

- first_amend_commit.png

- amend_commit.png

Summary:

- git commit --amend lets you modify your last commit, including its message or staged files.
- Useful for quick fixes before pushing to remote.
- Once pushed, amending requires force-push to update remote history.

Task 8 – Reverting a Commit (Safe Undo on Remote Branch)

1. Make a change and commit it:
2. echo "temporary text" >> temp.txt
3. git add .
4. git commit -m "Added temporary text file"

   git push origin main

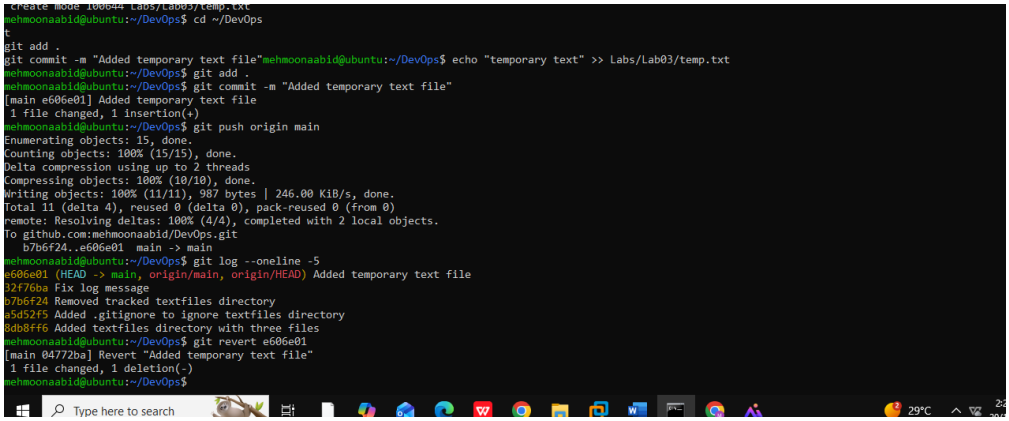
   - Save a screenshot as commit_temp_file.png.



5. Now revert that commit safely (do not delete it):
6. git log --oneline

   git revert <commit-hash>

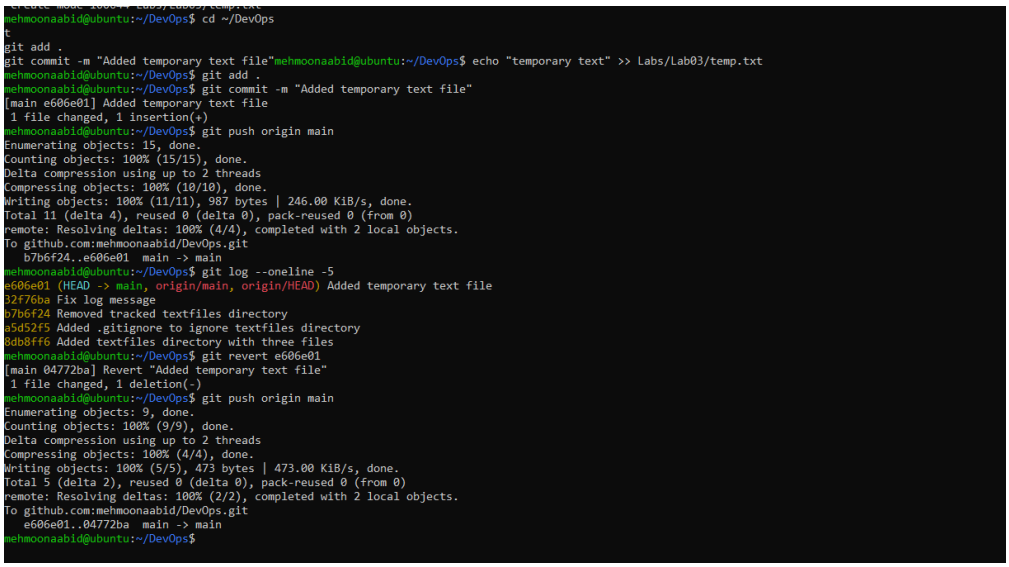
   - Save a screenshot as revert_commit.png.



7. Push the revert commit:

   git push origin main

   - Save a screenshot as revert_push.png.



📸 Screenshot Required:

- commit_temp_file.png
- revert_commit.png
- revert_push.png

Summary:

- git revert <hash> creates a new commit that undoes changes from a specific commit.
- It does not rewrite history, making it safe for shared branches.

- Always use revert (not reset) for undoing changes on remote branches.

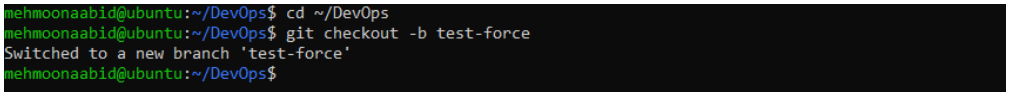
Task 9 – Force Push (With Caution)

⚠ Use this only in your own branch (never on main or develop).

1. Create a new branch:

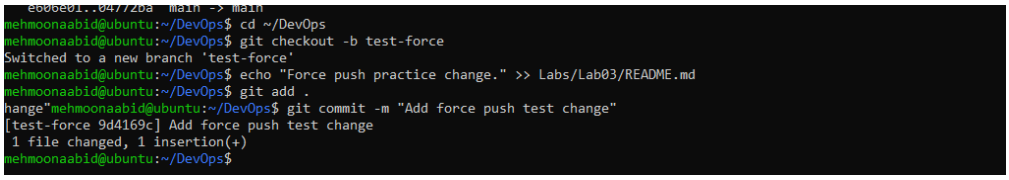
   git checkout -b test-force

   - Save a screenshot as new_branch.png.



2. Make and commit a small change.

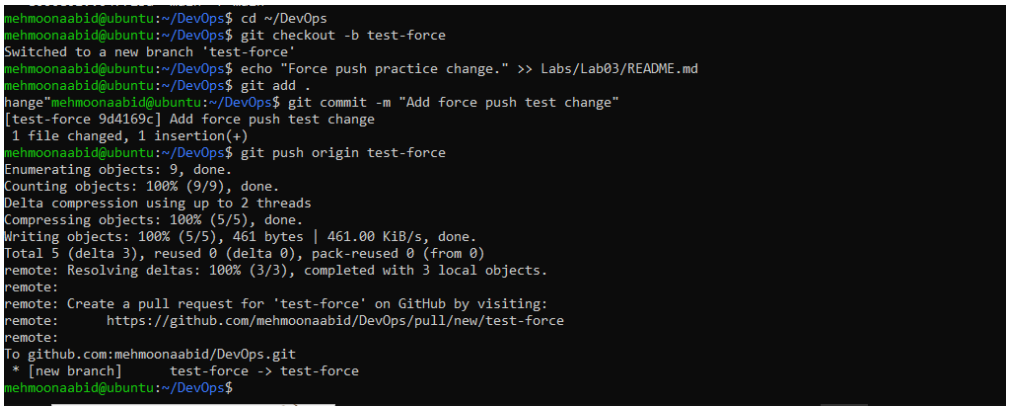
   - Save a screenshot as force_commit.png.



3. Push it:

   git push origin test-force

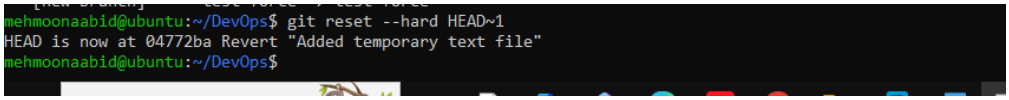
   - Save a screenshot as push_force_branch.png.



4. Perform a hard reset:

   git reset --hard HEAD~1

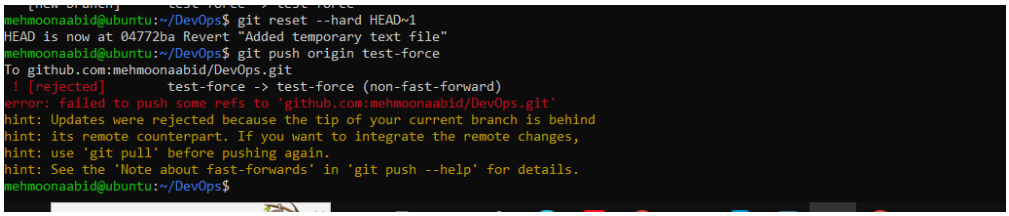
   - Save a screenshot as hard_reset_force.png.



5. Push again

   git push origin test-force

   - Rejected by remote repository
   - Save a screenshot as normal_push.png.



6. Now force-push to remote:

   git push origin test-force --force

   - Save a screenshot as force_push.png.



📸 Screenshot Required:

- new_branch.png
- force_commit.png
- push_force_branch.png
- hard_reset_force.png
- normal_push.png
- force_push.png

Summary:

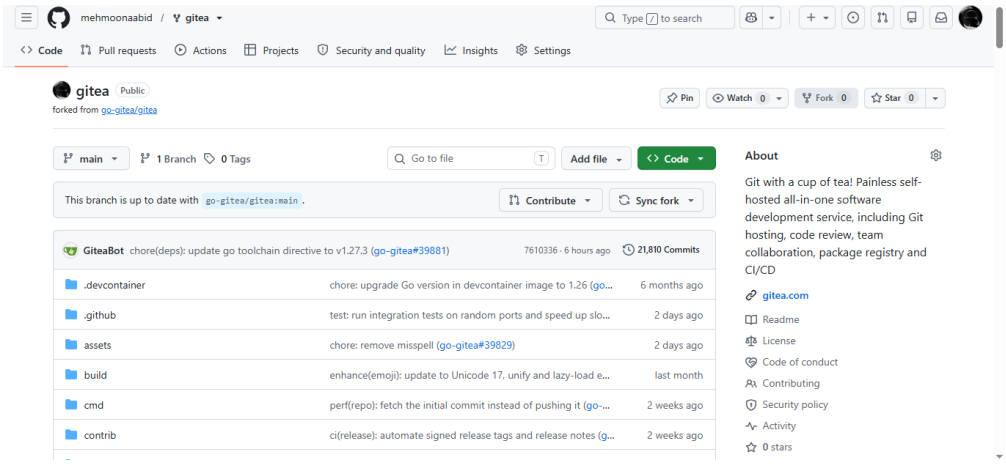
- Force push (git push --force) rewrites remote branch history, overwriting changes.
- Only use on non-shared branches to avoid disrupting collaborators.
- Always double-check before force-pushing.

Task 10 – Running Gitea in GitHub Codespaces via Docker Compose

This task will guide you through forking a repository, running a full web application in a GitHub Codespace, and interacting with Gitea (a self-hosted Git service) inside Codespaces.

Steps

1. **Fork the Gitea Repository**
    - Go to Gitea.
    - Click the **Fork** button at the top right and fork it into your own GitHub account.
    - Uncheck **Copy the main branch only** so that the codespace branch is also included, then click **Create fork**.
    - Save a screenshot of your forked repository page as forked_gitea.png



2. **Open the Forked Repo in GitHub Codespaces**
    - On your forked repo, click **Code** > **Codespaces** > **Create codespace on main**.
    - Save a screenshot of the Codespace loading as codespace_loading.png.



Setting up your codespace

3. **Switch to the codespace Branch and Start Gitea**

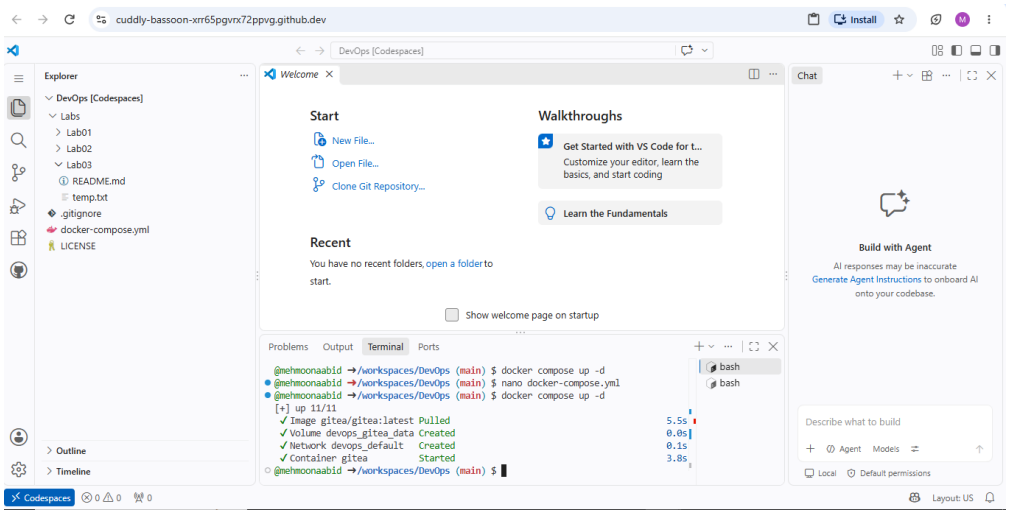
   - Before running Gitea, switch to the **codespace** branch in the Codespace terminal:

   git checkout codespace

   - After switching successfully, start Gitea with Docker Compose

   docker compose up -d

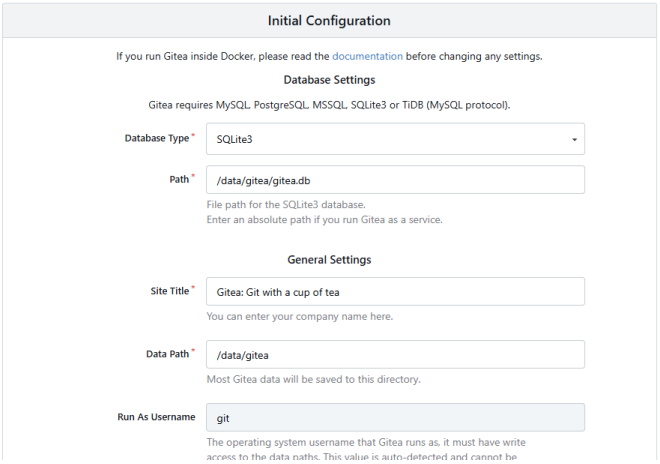
   - Wait for the containers to start.
   - Save a screenshot of the terminal showing the containers running as docker_up.png.



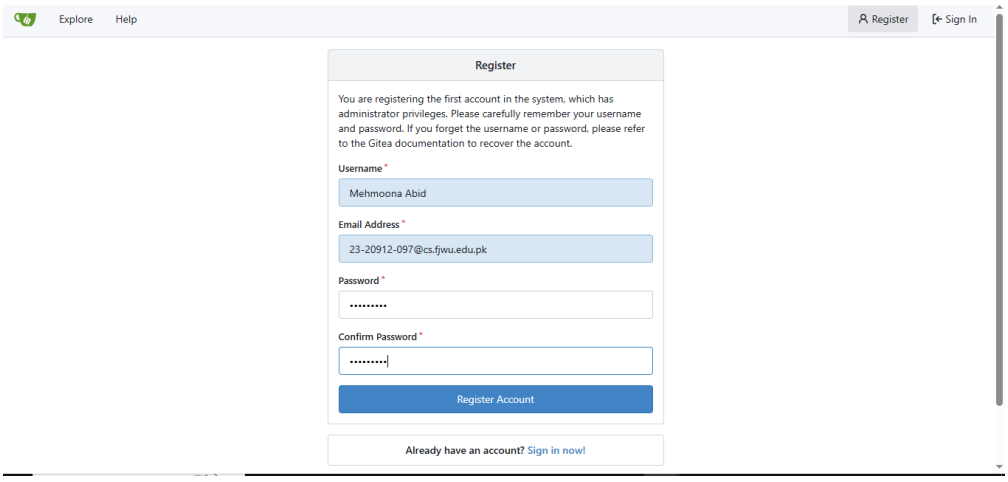
4. **Access Gitea Web Interface**

   - In Codespaces, forward **port 3000** (see Codespaces port forwarding UI).
   - Change the Visibility of **port 3000** from **private** to **public**.

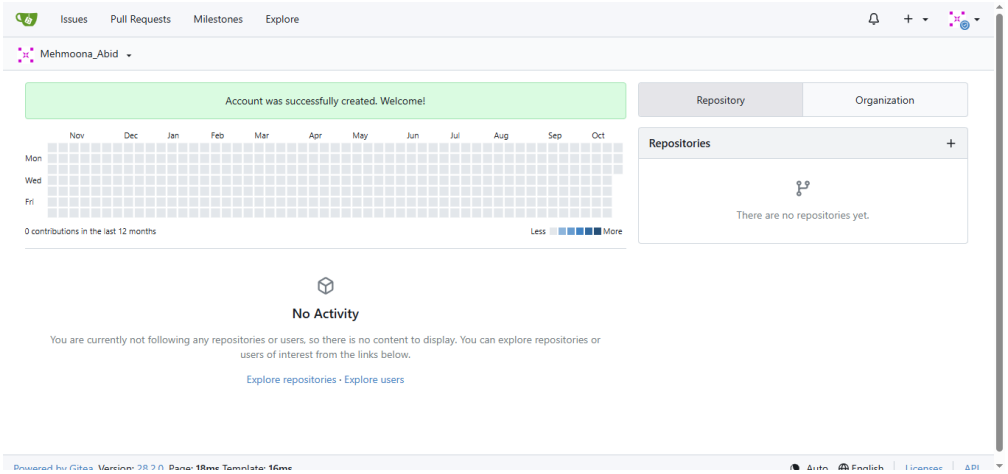
- Click the forwarded port 3000 link to open Gitea in your browser tab.
- Save a screenshot of the Gitea install page as gitea_install_page.png.



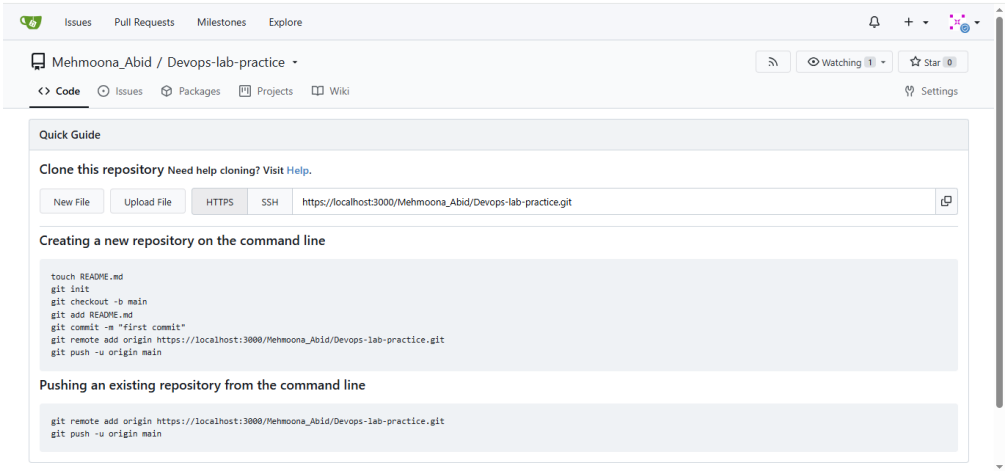
5. **Install Gitea**
   - Fill out the installation form, providing an **admin username** and **password** (e.g., admin / yourpassword).
   - Save a screenshot of the completed setup (show the admin account setup) as admin_setup.png.



6. **Log In to Gitea**
   - Use your admin credentials to log in.
   - Save a screenshot of the Gitea dashboard after login as gitea_dashboard.png.



7. **Create a New Repository in Gitea**
    - Click **New Repository** or **+ > New Repository** in Gitea.
    - Fill out the repository details and create it.
    - Save a screenshot of the new repository page as gitea_new_repo.png.



📸 **Screenshot Required:**

- forked_gitea.png
- codespace_loading.png
- docker_up.png
- gitea_install_page.png
- admin_setup.png
- gitea_dashboard.png

- gitea_new_repo.png

Summary:

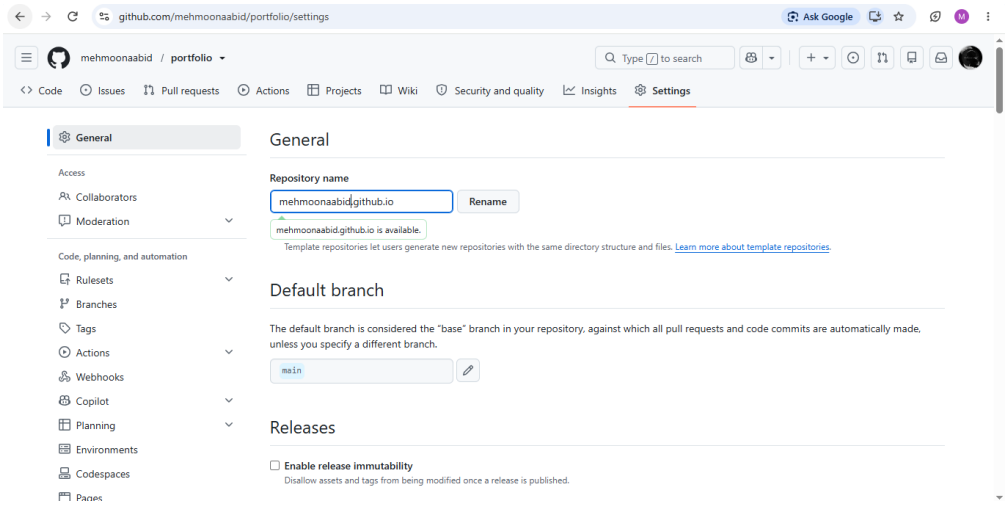
- You forked a repo and ran a real-world application using Docker Compose in Codespaces.
- You installed and configured a self-hosted Git service (Gitea) from scratch.
- You practiced using both GitHub and Gitea to create and manage repositories.

Task 11 – Creating a GitHub Pages Portfolio Site

This task will guide you through creating a personal portfolio/static site using GitHub Pages.

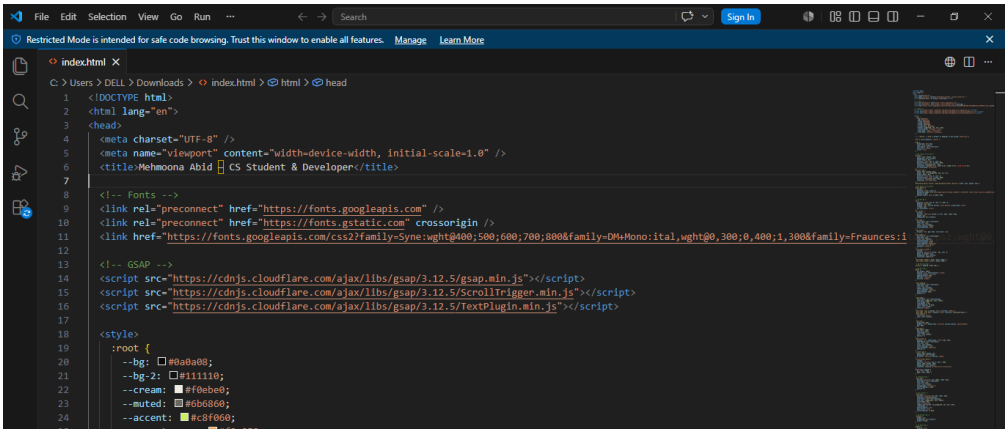
Steps

1. **Create a GitHub Pages Repository**
    - Go to GitHub and create a new repository named <your-username>.github.io
      (e.g., if your username is WaqasSaleem97, the repo should be WaqasSaleem97.github.io).
    - Make sure the repository is **public**.
    - Save a screenshot of the newly created repository as github_pages_repo.png.



2. **Add Static Website Code**

   - Prepare a simple static website (e.g., your CV or portfolio) in HTML/CSS/JS by using the sample code provided in the repository Iqra Bashir CV.
   - Put your files in a local folder.
   - Save a screenshot of your local files as local_static_site.png.



3. **Push the Files to GitHub**

   - Initialize git in your folder (if not already):
   - git init

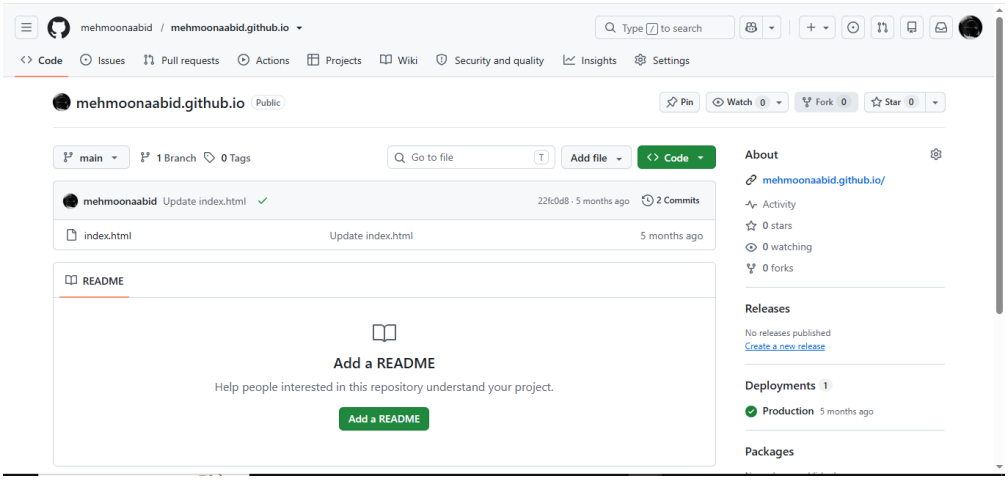
   git remote add origin git@github.com:<your-username>/<your-username>.github.io.git

   - Add and commit your files:
   - git add .

- git commit -m "Add portfolio site for GitHub Pages"

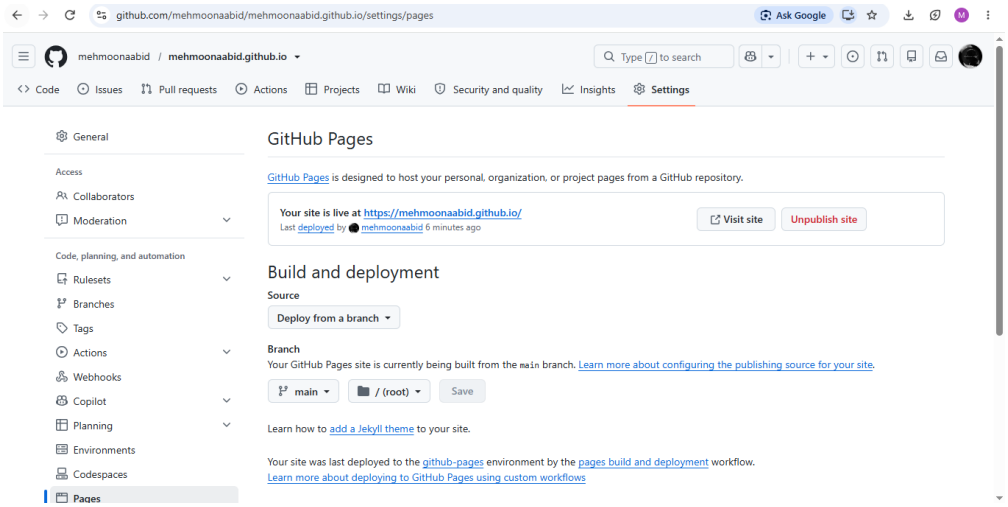
git push -u origin main

- Save a screenshot of your terminal showing a successful push as push_static_site.png.



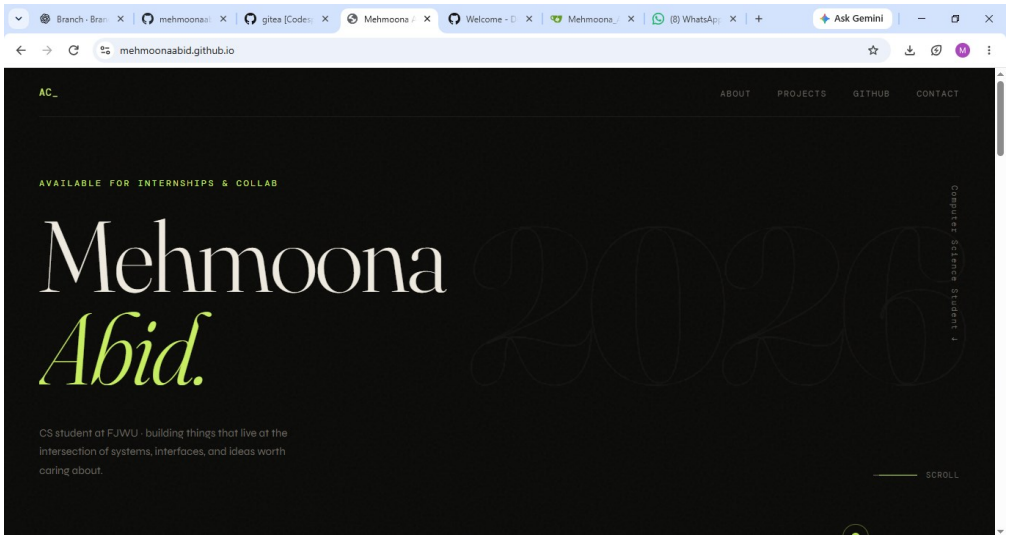
4. **Check GitHub Pages Settings**

- Go to your repository on GitHub.
- Click **Settings** > **Pages**.
- Confirm that your site is published and see the link to your live site (e.g., https://your-username.github.io).
- Save a screenshot of the Pages settings as github_pages_settings.png.



5. **Visit Your Live Site**

- Open your GitHub Pages site in the browser.
- Save a screenshot of your live site as live_site.png.



📸 **Screenshot Required:**

- github_pages_repo.png
- local_static_site.png
- push_static_site.png
- github_pages_settings.png
- live_site.png